

Overview

Track 16.2 – TypeScript for AI Applications · 16-ai-tech-stack/tracks/typescript/README.md

16.2 — TypeScript for AI Applications

Where you are: Track 2 of the AI tech-stack module (16). **Time:** ~12–15 hours · **Prereq:** working JavaScript, some TypeScript, and a rough idea of what an LLM API call looks like (../04-llm/).

Python trains and serves the model; TypeScript ships the product. Once the model is behind an HTTP endpoint, the interesting engineering moves to the application layer — streaming, tool execution, cancellation, edge deployment, and validating output from a system that has never heard of your type definitions. This track is about that layer, and about the one lesson that everything else hangs off: **a TypeScript type does not exist at runtime, and the model has never seen your tsconfig.**

Learning objectives

By the end of this track you can:

1. Model an LLM's message, content-block and streaming-event protocol as discriminated unions, and make `tsc` fail your build when a provider adds a variant you do not handle.
2. Define a tool once in Zod and derive all three artifacts from it — the static type (`z.infer`), the runtime validator (`safeParse`), and the JSON Schema you put on the wire (`z.toJSONSchema`).
3. Reject a malformed tool call from a model without throwing, and hand the model a repair message it can act on.
4. Write an SSE parser that survives hostile chunk boundaries — split UTF-8 sequences, frames straddling chunks, unflushed buffers — and explain why the naive version passes every test on localhost.
5. Cancel a generation end to end with `AbortController`, from a browser click through to the provider request, and handle a mid-stream `error` event on an already-200 response.
6. Choose correctly between the Vercel AI SDK, `@anthropic-ai/sdk`, and the Claude Agent SDK, and say what each one will and will not do for you.
7. Deploy to an edge runtime knowing exactly what breaks there, and configure a `tsconfig` whose strictness earns its keep.

Suggested path

#	Do this	File	Time
1	Read the core concepts	notes/01-core-concepts.md	2h
2	Run the tool-calling example	code/typed-tool-calling.ts	1h
3	Read streaming, SDKs and runtimes	notes/02-streaming-runtimes-and-hygiene.md	2h
4	Run and break the stream parser	code/streaming-parser.ts	1.5h
5	Do the exercises	lab/EXERCISES.md	6h
6	Skim the primary docs	papers/PAPERS.md	1.5h
7	Present it back	slides/ (typescript.pptx)	1h

Setup for steps 2 and 4 is in code/README.md — `npm install` and two `npx tsx` commands. No API key, no network.

The 16 terms you must own

discriminated union · narrowing · exhaustiveness check · structural typing · unknown vs any · satisfies · `z.infer` · `safeParse` · JSON Schema · async iterator · `ReadableStream` · SSE frame · backpressure · `AbortController` · partial UTF-8 · edge runtime

Exit check

Ship a single `POST /api/chat` route that:

1. takes a message list, calls a model, and streams tokens back over SSE;
2. exposes one Zod-defined tool, validates every call against it, and returns a `tool_result` with `is_error: true` — never a thrown exception — when the model gets the arguments wrong;
3. survives a client disconnect: the provider request is aborted and the log line says "cancelled", not "error";
4. passes `tsc --noEmit` with `strict`, `noUncheckedIndexedAccess` and `exactOptionalPropertyTypes` on;
5. has a test that replays a recorded SSE response at 1-byte chunks and asserts byte-identical output to the 4096-byte replay.

If point 5 passes, you have beaten the bug that ships in most AI products.